

Project Design Report

1. Cover Page

Rajshahi University of Engineering & Technology (RUET), Rajshahi-6204 Department of Computer Science & Engineering

Course	CSE 3206 – Software Engineering Sessional
Lab	Lab 2 – Software Process Models, Requirement Analysis & MVP Development
Project No.	23
Project Title	Smart Irrigation System
Process Model	Prototype
Group	[your group / section]
CO / PO	CO2 / PO3
Submitted To	[course teacher]
Date	[date]

2. Team Information

Member	Name	Roll	Responsibility	Branch
1	Masrur Mahin (Lead)	[roll]	Database, login/register, server setup, report	member1-auth
2	[Name]	[roll]	Field management and sensor readings	member2-fields
3	[Name]	[roll]	Pump control, auto irrigation, dashboard UI	member3-irrigation

Repository: <https://github.com/<username>/smart-irrigation>

3. Project Title

Smart Irrigation System — a web application that stores soil-moisture readings from each field and turns the water pump on or off, manually or automatically.

4. Problem Statement

Farmers usually irrigate by habit: the pump runs for a fixed time whether the soil needs water or not. This wastes water and diesel, damages the crop when it is over or under watered, and leaves no record of how much water each field received. The client wants a simple system that stores moisture readings per field, decides when irrigation is needed, controls the pump and keeps a history of every irrigation.

5. Project Objectives

1. Let a user register, log in and manage their own fields.
2. Store soil moisture, temperature and humidity readings for each field.
3. Compare each reading with the field’s moisture threshold.
4. Turn the pump ON or OFF manually or automatically.
5. Keep a history of every irrigation action with its cause.
6. Show everything on one simple dashboard.

6. Stakeholder Analysis

Stakeholder	Interest
Farmer (primary user)	Wants to water the right field at the right time and save cost
Farm manager	Sets moisture thresholds and reviews irrigation history
Field technician	Installs and maintains the sensors and the pump controller
Development team (Group 23)	Designs, builds and maintains the software
Course teacher	Evaluates the engineering method and documentation

7. Project Scope

In scope: user registration and login, field CRUD, sensor reading storage, a sensor simulator, threshold-based automatic pump control, manual pump control, and an irrigation history.

Out of scope: real sensor hardware and firmware, SMS or mobile notifications, weather forecast integration, payment, and a native mobile app.

8. Functional Requirements

ID	Requirement
FR-01	The system shall allow a new user to register with name, email and password.
FR-02	The system shall allow a registered user to log in and log out.
FR-03	The system shall show each user only their own fields.
FR-04	The system shall allow a user to add a field with name, crop, area and moisture threshold.
FR-05	The system shall list all fields of the user with their latest reading.
FR-06	The system shall allow a user to delete a field.
FR-07	The system shall store soil moisture, temperature and humidity readings for a field.
FR-08	The system shall reject a moisture value outside the range 0-100.
FR-09	The system shall generate simulated sensor readings while real hardware is unavailable.
FR-10	The system shall mark a field as “dry” when its latest moisture is below its threshold.
FR-11	The system shall allow a user to turn the pump ON or OFF manually for any field.
FR-12	The system shall prevent turning the pump ON when it is already ON.
FR-13	The system shall automatically start the pump for every field whose moisture is below its threshold.
FR-14	The system shall automatically stop the pump when moisture reaches the threshold.
FR-15	The system shall record every pump action with the field, action, cause (manual or auto) and moisture.

9. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Usability	A new user shall be able to add a field and start the pump without training.
NFR-02	Performance	Any page or API request shall respond within 2 seconds for up to 50 fields.
NFR-03	Security	Passwords shall be stored only as bcrypt hashes, never as plain text.
NFR-04	Security	Every field and pump operation shall be blocked unless the user is logged in.
NFR-05	Reliability	A missing sensor reading on one field shall not stop the other fields from working.
NFR-06	Maintainability	Code shall be separated into server, database and route files so a module can be changed independently.
NFR-07	Portability	The system shall run on Windows, Linux or macOS with Node.js and PostgreSQL installed.
NFR-08	Data integrity	Deleting a field shall also delete its readings and logs (foreign key cascade).
NFR-09	Auditability	Every pump action shall be permanently stored in the log table.
NFR-10	Responsiveness	The interface shall work on a mobile screen as well as a laptop.

10. User Stories

ID	User story	Acceptance
US-01	As a farmer, I want to log in so that only I can see my fields.	Wrong password is rejected; the dashboard opens only after a successful login.
US-02	As a farmer, I want to add my field with a moisture threshold so that the system knows when it is dry.	The new field appears in the table with its threshold.
US-03	As a farmer, I want to see the latest moisture of every field so that I know which one needs water.	The table shows the latest reading and a “dry” or “ok” status.
US-04	As a farmer, I want to turn the pump on from my phone so that I do not have to walk to the field.	Pressing “Turn ON” changes the pump status and adds a MANUAL entry to the history.
US-05	As a farm manager, I want the system to water dry fields by itself so that irrigation does not depend on someone being present.	“Run auto irrigation” starts the pump on every field below its threshold and logs it as AUTO.

Use case: Automatic irrigation

Actor	Farmer / farm manager
Precondition	User is logged in and at least one field has a reading
Main flow	1. User clicks “Run auto irrigation”. 2. System reads the latest reading of each field. 3. System compares moisture with the threshold. 4. Pump is started or stopped. 5. The action is written to the log. 6. Results are shown on screen.
Alternate flow	2a. No reading exists → the field is skipped with a message.
Postcondition	Pump status and history are updated

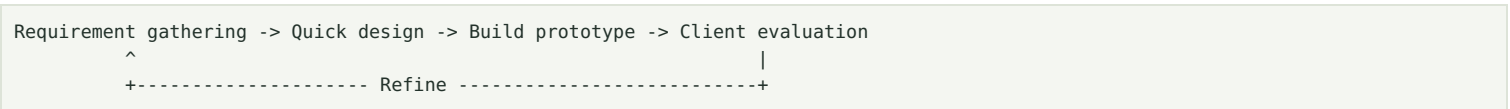
11. Assumptions and Constraints

Assumptions: each field has one moisture sensor and one pump; readings arrive over the network (simulated in this lab); one field belongs to one user.

Constraints: three students, one semester; no budget for hardware or hosting; must run locally with Node.js and PostgreSQL; the MVP must stay small so that later labs can add design patterns and testing.

12. Selected Software Process Model

Prototype model. We build a small working version, show it to the client, take their feedback, and improve it in the next round.



Round	What it contains
Prototype 1 (this lab)	Login, fields, readings, manual and automatic pump control
Prototype 2	Changes requested after the demonstration; alerts and reports
Prototype 3	Real sensor input, refactoring into design patterns, testing

13. Justification of the Process Model

1.

The client cannot describe the software in advance. A farmer cannot answer “what should the dashboard contain?”, but can easily answer “is this screen useful?” A prototype turns an unanswerable question into an answerable one.
2.

The interface is the risky part, not the logic. The irrigation rule is a simple comparison; the real risk is that the screen is confusing. Only a working screen in front of a real user can remove that risk.
3.

The hardware is not final. The sensor type and data format are undecided, so we replace the hardware with a simulator and keep building.
4.

Requirements will change. Thresholds, auto mode and the history table all changed while writing this report, and will change again after the demonstration.
5.

It fits a small student team. Each round is short, always produces something demonstrable, and needs no heavy process documents.

14. Comparison with Alternative Models

Waterfall — not suitable. Waterfall needs a complete, frozen requirement document before coding, and shows nothing to the client until the end. Our requirements are not known at the start, so a mistake in the requirement phase would only be discovered at delivery, when it is most expensive to fix.

Spiral — not suitable. Spiral is also prototype-based, but adds formal risk analysis in every cycle. That effort is worth it for large, expensive or safety-critical systems. Our project is a one-semester academic system with three students and no budget, so the risk analysis would cost more time than it saves.

Agile Scrum — not suitable here. Scrum needs daily stand-ups, sprint planning and a product owner who is always available. Our team meets around class schedules and the client is available only occasionally, so we could not follow Scrum honestly — and a half-followed Scrum is worse than a properly followed Prototype model.

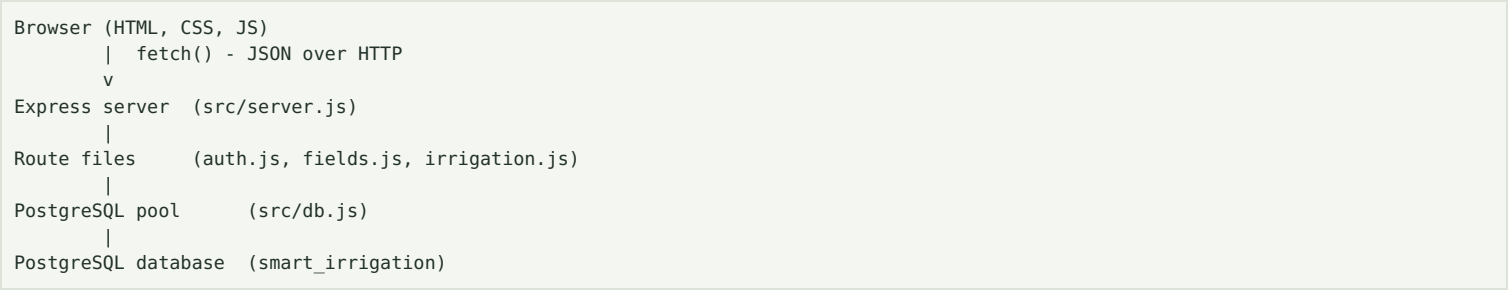
Model	Fit	Reason
Prototype	Selected	Requirements are discovered by showing a working screen
Incremental	Possible	Good for delivery, but assumes the increments are already known
Waterfall	Poor	Needs frozen requirements that do not exist
Spiral	Poor	Risk analysis is too heavy for this project size
Agile Scrum	Poor	Ceremonies do not fit the team’s and client’s availability

15. MVP Design Overview

Technology

Layer	Technology
Frontend	HTML, CSS, JavaScript (no framework)
Backend	Node.js with Express
Database	PostgreSQL
Login	express-session + bcryptjs

Architecture



Database tables

Table	Columns
users	id, name, email, password
fields	id, user_id, name, crop, area, threshold, pump_status
readings	id, field_id, moisture, temperature, humidity, recorded_at
logs	id, field_id, action, trigger_by, moisture, created_at

One user has many fields; one field has many readings and many logs (ON DELETE CASCADE).

API

Method	Path	Purpose
POST	/api/register, /api/login, /api/logout	Account and session
GET	/api/me	Logged-in user
GET, POST	/api/fields	List and add fields
DELETE	/api/fields/:id	Delete a field
POST, GET	/api/fields/:id/reading(s)	Save and read sensor data
POST	/api/simulate	Generate one reading per field
POST	/api/pump/:id/on , /api/pump/:id/off	Manual pump control
POST	/api/auto	Automatic irrigation for all fields
GET	/api/logs	Irrigation history

Irrigation rule

Condition	Action
No reading yet	Skip the field
Moisture < threshold and pump OFF	Start the pump, log as AUTO
Moisture ≥ threshold and pump ON	Stop the pump, log as AUTO
Otherwise	No change

Screens

1. **Login / Register** — one page with two tabs.
2. **Dashboard** — add-field form, field table (moisture, threshold, status, pump, actions), buttons for “Take sensor reading” and “Run auto irrigation”, and the irrigation history table.

16. GitHub Collaboration Evidence

Branch	Member	Pull request	Status
member1-auth	Masrur Mahin	PR #1	Merged
member2-fields	[Name]	PR #2	Merged
member3-irrigation	[Name]	PR #3	Merged

Insert screenshots of the branches, pull requests, commit history and network graph from the screenshots/ folder here.

17. Challenges Encountered

1. **Deciding what to leave out.** The first requirement list had weather forecasts and SMS alerts. Following the prototype model, we kept only what was needed to prove the idea.
2. **No sensor hardware.** Solved by writing a simulator that produces random readings, which can later be replaced by real sensor data on the same endpoint.
3. **Merge conflicts.** `server.js` was edited by all three members at first. We fixed this by splitting the routes into three separate files, one per member.
4. **Database setup differences.** Team members had different PostgreSQL passwords, so the connection settings were moved to environment variables with a `.env.example` file instead of being written inside the code.

18. Conclusion

The prototype fulfils the core idea of the project: a user can register their fields with a moisture threshold, store sensor readings, and let the system decide when the pump should run — with every action recorded. The Prototype model was the right choice because the uncertainty was in the requirements and the interface, not in the technology. The code is small and divided into clear files, so the next labs can add design patterns, testing and real sensors without rewriting it.